

沈阳航空航天大学

《模式识别》实验报告

题目：K-近邻算法与模型选择

院（系）： 人工智能学院

专 业： 物联网工程

班 级： 物联网2202

学 号： 223428010210

姓 名： 陈梓欣

指导教师： 刘翠微

1 任务描述

- A. 实验目的：通过编程和上机实验加深对监督学习的理解和认识，掌握讲授的模式分类算法。编程实现 K 近邻算法分类算法，并在给定的数据集上测试算法的性能。掌握基于交叉验证的模型选择方法，能够分析参数对算法性能的影响。
- B. 实验环境：PC 机，window 操作系统，python（包括 numpy、matplotlib、os 库）。
- C. 实验要求：
- (1) 掌握讲授的模式分类算法；
 - (2) 实现 K 近邻算法，并在给定的数据集上测试算法性能；
 - (3) 掌握基于交叉验证的模型选择方法；
 - (4) 编程实现 N-fold 交叉验证方法。

2 实验内容

(1) 数据集的载入与划分

在本实验中，首先从文件 `dataset.txt` 中载入数据集，共包含 800 行样本。每行由三列组成：前两列为样本的二维坐标，第三列为类别标签（1 表示正类样本，-1 表示负类样本）。

将前 500 行数据作为训练数据集（`traindata`），其余 300 行作为测试数据集（`testdata`）。在训练阶段可使用类别标签进行模型训练与验证。

通过 `numpy.loadtxt()` 函数读取数据，并使用 `matplotlib.pyplot.scatter()` 对样本的二维分布进行可视化，直观展示不同类别样本的分布特征。

(2) K-近邻算法的实现

采用欧氏距离作为样本间的距离度量。对每个测试样本，计算其到所有训练样本的距离，并选取距离最近的 K 个邻居。根据邻居的类别标签进行多数表决，确定测试样本的预测类别。

实验中设置 $K=1, 3, 5, 7, 9$ 五种取值，通过循环依次进行分类。为方便计算与结果比较，编写自定义函数实现 KNN 分类与精度评估。

可视化结果：

- 使用不同颜色绘制不同类别的样本点；
- 聚类中心用特殊符号标注（如五角星、十字或圆圈）。

(3) 5-fold 交叉验证

将训练数据集 `traindata` 及其对应的标签随机划分为 5 份 (fold)。每次循环中，取其中 1 份作为验证集，其余 4 份作为训练集，重复 5 次，保证每个样本都被验证一次。

对每次循环，在不同 K 值 (1,3,5,7,9) 下分别计算 KNN 分类正确率。最终输出每轮验证中各 K 值对应的准确率，并计算 5 次验证的平均正确率，用于模型选择。

(3) 模型选择与结果评估

根据 5-fold 交叉验证的平均正确率，选择分类性能最优的 K 值作为最终模型参数。然后，使用选定的最优 K 值在独立的测试数据集上进行分类测试，计算总体准确率，并与交叉验证结果进行比较。

通过分析不同 K 值下模型的准确率变化和最终测试性能，验证交叉验证在参数选择中的有效性和模型泛化能力。

3 实验结果和分析

A. 实验结果

(1) 载入数据集中的数据

载入数据集中的数据，并将训练集中的所有样本显示在二维图像中，红色表示正样本，蓝色表示负样本。原始数据加载效果如图 1 所示。



图 1 原始数据加载效果图

(2) 编程实现 K 近邻算法

实现 K 近邻分类算法。设置 $K=1,3,5,7,9$ ，输出在不同的 K 值下，测试数据集 testdata 的分类结果，并计算分类正确率，具体实现效果如图 2 所示。

KNN分类结果：

K = 1 : 准确率 = 99.00%
K = 3 : 准确率 = 99.00%
K = 5 : 准确率 = 99.33%
K = 7 : 准确率 = 99.33%
K = 9 : 准确率 = 99.00%

图 2 不同 k 值下 KNN 分类结果

(3) 编程实现 N-fold 交叉验证方法

载入数据集中的数据，将训练数据集 traindata 及相应的标签分成 5 份，进行 5-fold 交叉验证。循环 5 次，每次取一份数据作为测试数据，其余的数据作为训练数据，采用 K 近邻算法进行分类，取 K=1,3,5,7,9。不同的 K 取值下 5-Fold 交叉验证结果如图 3 所示。

```
===== 5-Fold 交叉验证结果 =====

Fold 1:
  K = 1 -> 准确率: 99.00%
  K = 3 -> 准确率: 99.00%
  K = 5 -> 准确率: 99.00%
  K = 7 -> 准确率: 99.00%
  K = 9 -> 准确率: 99.00%
-----

Fold 2:
  K = 1 -> 准确率: 98.00%
  K = 3 -> 准确率: 99.00%
  K = 5 -> 准确率: 99.00%
  K = 7 -> 准确率: 99.00%
  K = 9 -> 准确率: 98.00%
-----

Fold 3:
  K = 1 -> 准确率: 100.00%
  K = 3 -> 准确率: 100.00%
  K = 5 -> 准确率: 100.00%
  K = 7 -> 准确率: 100.00%
  K = 9 -> 准确率: 100.00%
-----

Fold 4:
  K = 1 -> 准确率: 99.00%
  K = 3 -> 准确率: 99.00%
  K = 5 -> 准确率: 99.00%
  K = 7 -> 准确率: 99.00%
  K = 9 -> 准确率: 99.00%
-----

Fold 5:
  K = 1 -> 准确率: 99.00%
  K = 3 -> 准确率: 98.00%
  K = 5 -> 准确率: 100.00%
  K = 7 -> 准确率: 98.00%
  K = 9 -> 准确率: 99.00%
-----
```

图 3 不同的 K 取值下 5-Fold 交叉验证结果

输出每次循环中，不同的 K 取值下 K 近邻算法的分类正确率。并根据 5 次循环的平均正确率选择 K 的最佳取值，最终结果如图 4 所示。

===== 各K值平均准确率 =====
K = 1: 平均准确率 = 99.00%
K = 3: 平均准确率 = 99.00%
K = 5: 平均准确率 = 99.40%
K = 7: 平均准确率 = 99.00%
K = 9: 平均准确率 = 99.00%

最佳K值为: 5
平均准确率最高为: 99.40%

图 4 最终所选 K 的最佳取值

B. 实验分析

(1) 不同 K 值下 K 近邻算法结果变化分析

在 K 近邻算法中，K 值的选择直接影响分类性能。

- 当 $K=1$ 时，模型仅依赖最近邻样本，容易受到噪声影响，表现为训练集准确率高但泛化能力差；
- 随着 K 增大至 3 和 5，算法对单个异常样本的敏感度降低，分类边界更平滑，通常能获得更高的平均准确率。
- 当 K 继续增大到 7 或 9 时，邻域范围过宽，可能包含不同类别的样本，导致分类边界模糊、准确率下降。

因此，K 值过小会造成过拟合，K 值过大则可能欠拟合。综合各轮交叉验证的结果，通常可观察到准确率在中等 K 值（如 $K=3$ 或 $K=5$ ）时达到较优平衡点。

(2) 根据 5-fold 交叉验证结果选择最优 K 值及结果一致性分析

5-fold 交叉验证通过多次划分训练集与验证集，评估不同 K 值下模型的平均性能，从而避免单一划分带来的偶然性。

实验中，通过计算 $K=1、3、5、7、9$ 在五次交叉验证中的平均准确率，可选择平均准确率最高的 K 值作为最优参数。通常，该 K 值能在训练数据上实现较好的泛化能力。

当将该最优 K 值应用于独立测试集时，若测试准确率与交叉验证平均准确率相近，说明模型具有良好的泛化性能；若差异较大，可能是训练样本代表性不足或存在过拟合现象。

因此，交叉验证不仅用于参数选择，也能有效评估模型稳定性与适用性。

(3) 实验结论

通过本次实验，我们在二维数据集上实现了 K-近邻 (KNN) 算法的分类与模型选择过程。

首先将训练数据集进行 5-fold 交叉验证，分别在 $K=1、3、5、7、9$ 时计算分类准确率。

实验结果表明，K 值对 KNN 分类性能影响显著：

- 当 K 过小（如 $K=1$ ）时，模型对噪声较为敏感，容易过拟合；
- 当 K 过大时，分类边界过于平滑，导致欠拟合。

综合五次交叉验证的平均正确率可见，K 在中等取值（如 $K=3$ 或 $K=5$ ）时性能最佳，模型稳定性和泛化能力较强。

最终，以交叉验证得到的最优 K 值在测试集上验证，测试结果与交叉验证结果基本一致，说明 5-fold 交叉验证能够有效指导 KNN 模型参数选择，并提升模型在未知数据上的分类性能。

4 附录

加载数据集

```
data = np.loadtxt("data/dataset.txt")
```

```
print("数据集载入完成, 共 {} 行".format(data.shape[0]))
```

分割训练集与测试集

```
train_data = data[:500, :] # 前 500 行
```

```
test_data = data[500:, :] # 后 300 行
```

提取训练数据坐标和标签

```
x_train = train_data[:, 0]
```

```
y_train = train_data[:, 1]
```

```
labels_train = train_data[:, 2]
```

绘制训练数据分布图

```
plt.figure(figsize=(7,6))
```

```
plt.title("Train Data Visualization", fontsize=13)
```

绘制红色与蓝色散点

```
plt.scatter(x_train[labels_train == 1],  
            y_train[labels_train == 1],  
            color='red', s=25, label='Label 1')
```

```
plt.scatter(x_train[labels_train == -1],  
            y_train[labels_train == -1],  
            color='blue', s=25, label='Label -1')
```

设置坐标轴标签

```
plt.xlabel("X-axis")
```

```
plt.ylabel("Y-axis")
```

显示图例、网格

```
plt.legend()
```

```
plt.grid(True, linestyle='--', alpha=0.6)
```

```
plt.show()
```

```

# 提取训练集与测试集数据

x_train = train_data[:, 0:2]    # 前两列为坐标

y_train = train_data[:, 2]      # 第三列为类别标签

x_test = test_data[:, 0:2]
y_test = test_data[:, 2]

# 定义欧式距离函数

def euclidean_distance(x1, x2):
    """计算两个样本之间的欧式距离"""
    return np.sqrt(np.sum((x1 - x2) ** 2))

# 定义 KNN 分类器

def knn_predict(x_train, y_train, x_test, K):
    y_pred = []
    for x in x_test:
        # 计算该测试样本与所有训练样本的距离

        distances = [euclidean_distance(x, x_t) for x_t in x_train]

        # 按距离排序, 取最近 K 个

        nearest_indices = np.argsort(distances)[:K]
        nearest_labels = y_train[nearest_indices]

        # 多数表决法

        label = np.sign(np.sum(nearest_labels))
        y_pred.append(label)
    return np.array(y_pred)

# 测试不同 K 值下的分类结果

K_values = [1, 3, 5, 7, 9]

print("KNN 分类结果: \n")

for K in K_values:
    y_pred = knn_predict(x_train, y_train, x_test, K)
    accuracy = np.mean(y_pred == y_test)

    print(f"K = {K} : 准确率 = {accuracy*100:.2f}%")

```

```

# 提取训练集数据

X = train_data[:, 0:2]    # 坐标

y = train_data[:, 2]      # 标签 (1 或 -1)

# 定义欧式距离函数
def euclidean_distance(a, b):
    """计算两个样本之间的欧式距离"""
    return np.sqrt(np.sum((a - b) ** 2))

# 定义 KNN 分类器
def knn_predict(X_train, y_train, X_test, K):
    """KNN 分类预测"""
    y_pred = []
    for x in X_test:
        # 计算与所有训练样本的距离
        distances = [euclidean_distance(x, x_t) for x_t in X_train]
        # 找到最近 K 个样本
        nearest_idx = np.argsort(distances)[:K]
        nearest_labels = y_train[nearest_idx]
        # 多数表决
        label = np.sign(np.sum(nearest_labels))
        y_pred.append(label)
    return np.array(y_pred)

# 实现 5 折交叉验证
K_values = [1, 3, 5, 7, 9]
fold_num = 5
fold_size = len(X) // fold_num

# 存储平均准确率
avg accuracies = {K: [] for K in K_values}

print("===== 5-Fold 交叉验证结果 =====\n")

```

```

for fold in range(fold_num):
    # 定义当前折的验证集范围

    start = fold * fold_size
    end = (fold + 1) * fold_size

    # 划分验证集与训练集

    X_val = X[start:end]
    y_val = y[start:end]
    X_train = np.concatenate((X[:start], X[end:]), axis=0)
    y_train = np.concatenate((y[:start], y[end:]), axis=0)

    print(f"Fold {fold+1}:")

    # 对不同 K 进行测试

    for K in K_values:
        y_pred = knn_predict(X_train, y_train, X_val, K)
        acc = np.mean(y_pred == y_val)
        avg accuracies[K].append(acc)

        print(f"  K = {K} -> 准确率: {acc*100:.2f}%")

    print("-" * 40)

# 计算每个 K 的平均准确率

print("===== 各 K 值平均准确率 =====")

for K in K_values:
    mean_acc = np.mean(avg accuracies[K])

    print(f"K = {K}: 平均准确率 = {mean_acc*100:.2f}%")

# 找出最佳 K 值

best_K = max(K_values, key=lambda k: np.mean(avg accuracies[k]))
best_acc = np.mean(avg accuracies[best_K])

print("\n 最佳 K 值为:", best_K)

print(f"平均准确率最高为: {best_acc*100:.2f}%")

```